

Tarea 3: Análisis, Desarrollo y Refinamiento

Taller de Python para Ciencia de Datos

1. Enunciado

El objetivo de esta actividad es desarrollar una aplicación en **Jupyter Notebook** que articule, de manera narrativa y autocontenida, el ciclo completo de un proyecto de Ciencia de Datos. Para ello, se deberá seleccionar un conjunto de datos (ya preparado o que requiera un preprocesamiento simple) idóneo para la aplicación de modelos de pronóstico basados en técnicas de regresión.

El desarrollo se debe realizar en **Python**, utilizando las librerías revisadas en el curso (**NumPy**, **Pandas**, **Matplotlib**, **Seaborn**, **SciPy** y **Scikit-Learn**, entre otras). La propuesta debe cumplir con los siguientes requerimientos:

- **Selección de datos:** Identificar un conjunto de datos apropiado para un problema de aprendizaje supervisado (regresión).
- **Definición de objetivos:** Declarar explícitamente el propósito del análisis. El flujo de trabajo en el **Notebook** debe ser autoexplicativo, alternando bloques de código con celdas **Markdown** bien estructuradas.
- **Análisis Exploratorio de Datos (EDA):** Evaluar los atributos y variables críticas para el modelamiento. Para esto, se sugiere integrar herramientas como:
 - Atributos y métodos de exploración: `df.dtypes`, `df.corr()`, `df.describe()`, `df.value_counts()`, `df.groupby()`, `df.pivot_table()` entre otros.
 - Visualizaciones: `sns.regplot()` y `sns.boxplot()`.

- **Evaluación y Refinamiento del Modelo:** Implementar estrategias para diagnosticar y mitigar problemas de subajuste (*underfitting*) o sobreajuste (*overfitting*). Considere el uso de:
 - División de datos: `train_test_split()` configurando adecuadamente el tamaño de muestra (`test_size`).
 - Validación cruzada: `cross_val_score()` y `cross_val_predict()`.
 - Métricas de rendimiento: Error Cuadrático Medio (MSE) y Coeficiente de Determinación (R^2).
 - Regularización: Modelos `Ridge()` y/o `Lasso()`.
 - Optimización de hiperparámetros: Búsqueda en grilla mediante `GridSearchCV()`.
- **Ciclo de Vida del Proyecto:** Estructurar el código respetando las etapas estándar de un flujo analítico:
 1. Lectura e ingesta del conjunto de datos.
 2. Limpieza y preprocesamiento de datos.
 3. Configuración, entrenamiento y ajuste del modelo.
 4. Generación de predicciones o pronósticos.
 5. Evaluación e interpretación del rendimiento.
 6. Refinamiento y optimización.
 7. Visualización de resultados finales.

2. Metodología de Aplicación

El entregable final debe ser un archivo `.ipynb` (Jupyter Notebook). Se evaluará la capacidad de “contar una historia” (*storytelling*) en torno a los datos; por lo tanto, el uso de narrativas claras en **Markdown** que justifiquen cada decisión técnica (por qué se elige una variable, por qué se descarta un modelo, etc.) es tan importante como la correcta ejecución del código Python.

3. Evaluación

El proyecto debe ser de autoría estrictamente original y satisfacer la totalidad de los requerimientos. Cada equipo/estudiante deberá realizar una defensa o testeo práctico del código junto al profesor para validar su funcionamiento y presentación.

La ponderación de la nota final se distribuye de la siguiente manera:

- **Avance (20 %):** Trabajo efectivo y cumplimiento de hitos desarrollado durante las sesiones semanales.

- **Interfaz y Formato (10 %):** Presentación estética del `Notebook`, claridad en las instrucciones escritas y un correcto manejo de excepciones/errores en la ejecución.
- **Calidad del Código (30 %):** Originalidad, modularización (evitar código redundante), uso correcto de funciones, métodos y buenas prácticas de programación.
- **Funcionalidad (40 %):** Nivel de cobertura y éxito en la implementación técnica de los requerimientos solicitados en el enunciado.

4. Entregas

La entrega de los archivos correspondientes se realizará a través de la plataforma institucional **EVA**. La fecha límite estará estipulada en el Syllabus de la asignatura o conforme a lo acordado formalmente en las sesiones de clase.